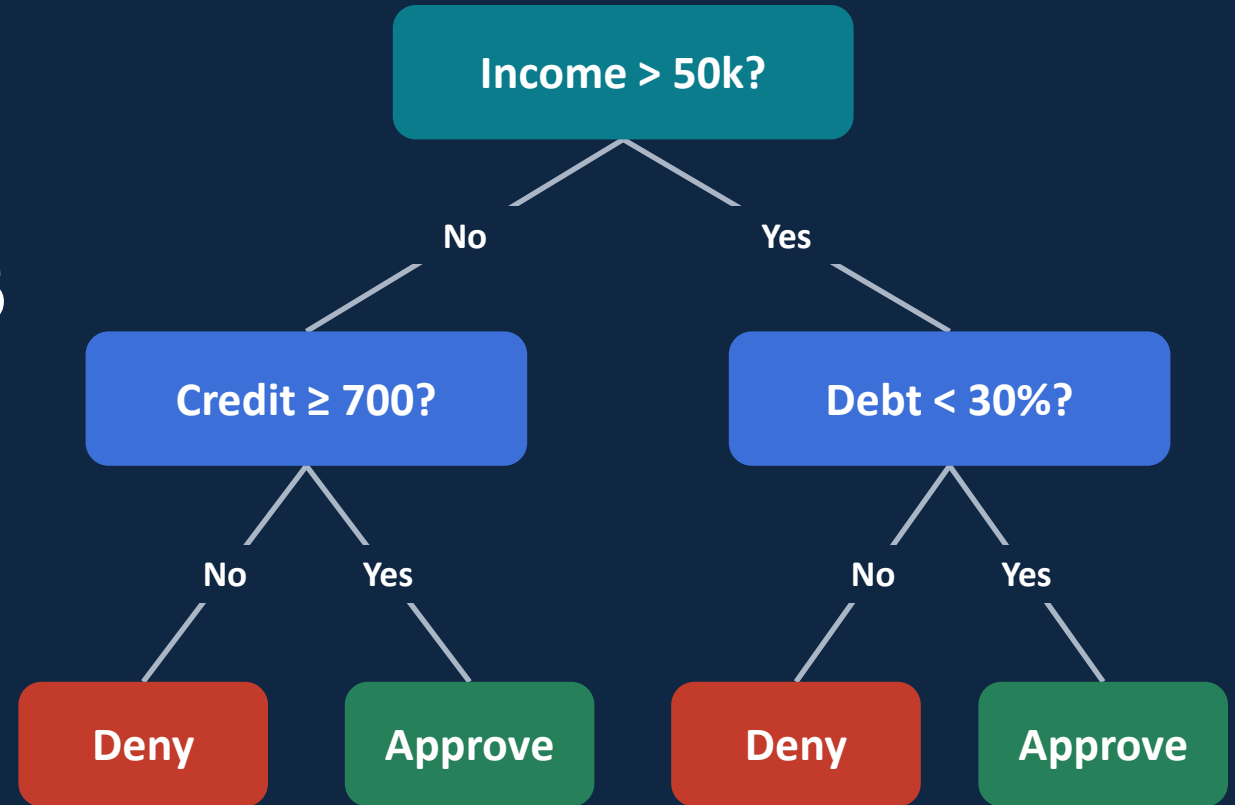


MACHINE LEARNING · LECTURE

Decision Trees

From Intuition to Mathematical Splitting
and Python Implementation



Learning objectives

INTRO

1

Explain how a tree makes predictions

2

Identify nodes, branches and leaves

3

Describe recursive binary splitting

4

Calculate Gini impurity and entropy

5

Compute information gain for a split

6

Train a tree with scikit-learn

7

Diagnose overfitting

8

Choose sensible hyperparameters

Why do we need decision trees?

INTUITION

Loan applicant

Income

\$65,000

Credit score

710

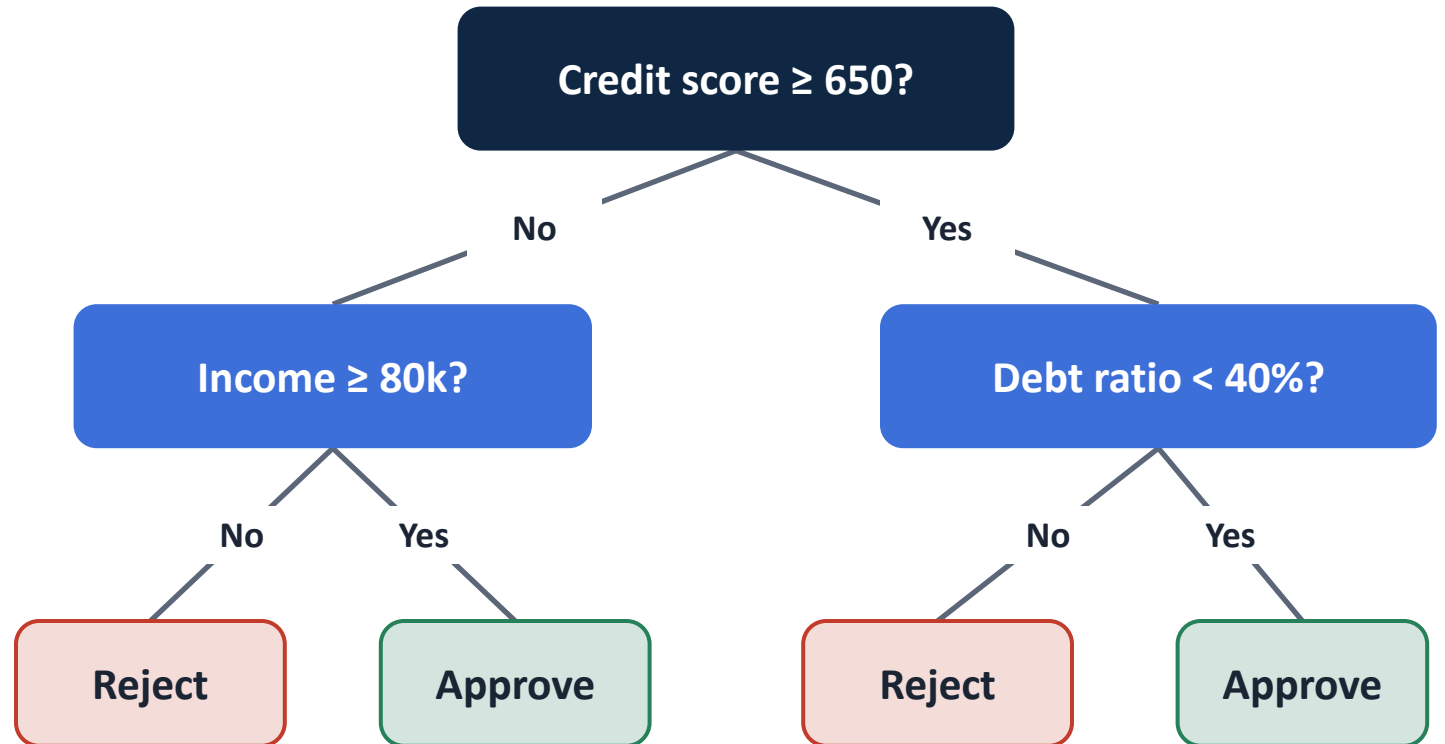
Existing debt

\$12,000

Employment

Full-time

Should we approve this loan?

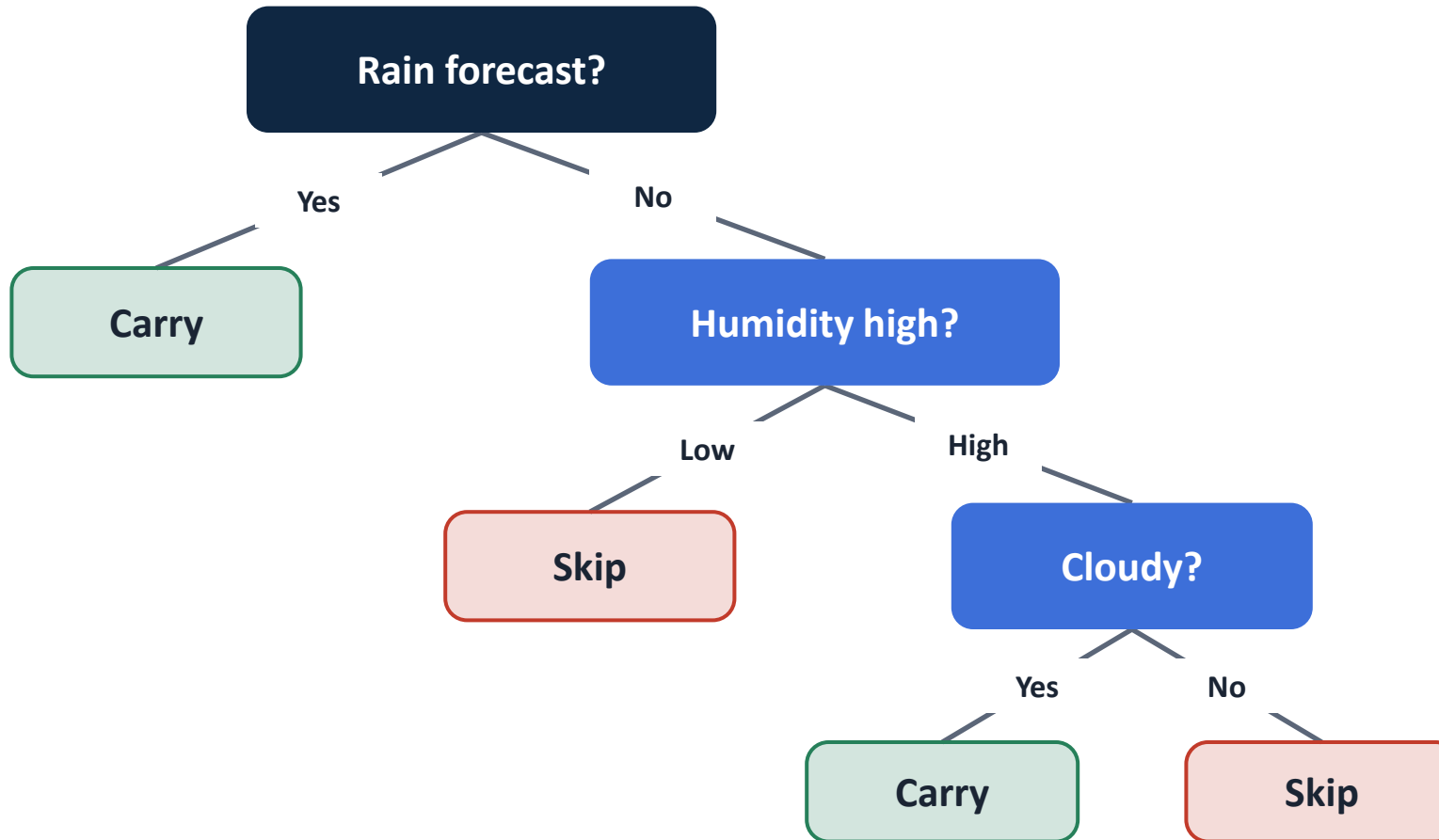


Three simple questions → one decision

Human decisions are rule-based

INTUITION

Should I carry an umbrella?



Each path = one rule

IF rain forecast

THEN carry

IF no rain AND humid AND
cloudy

THEN carry

OTHERWISE

THEN skip

What is a decision tree?

STRUCTURE

A supervised, non-parametric model that predicts by **recursively partitioning** the feature space with simple feature–threshold tests.

Root node

First question; has no parent

Internal node

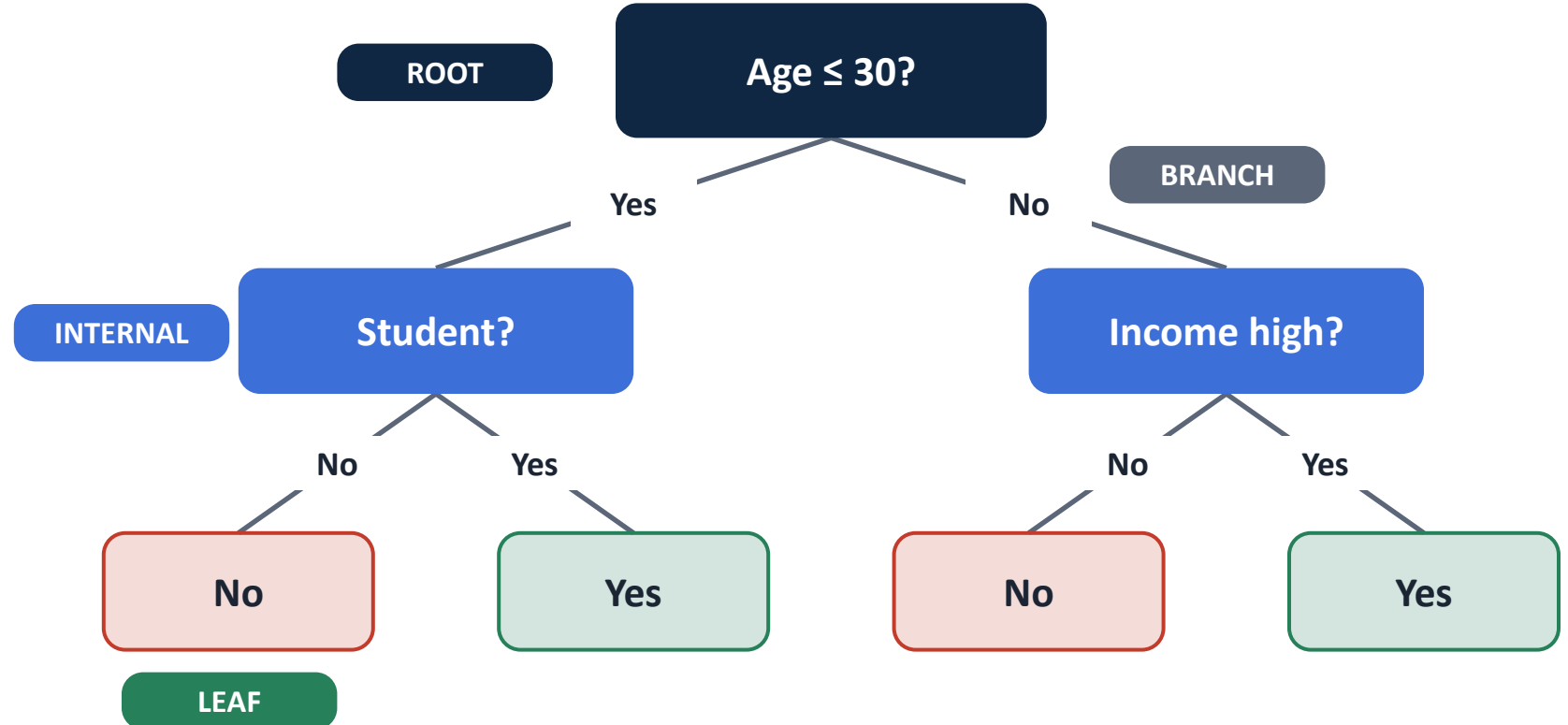
Tests one feature

Branch

Outcome of a test

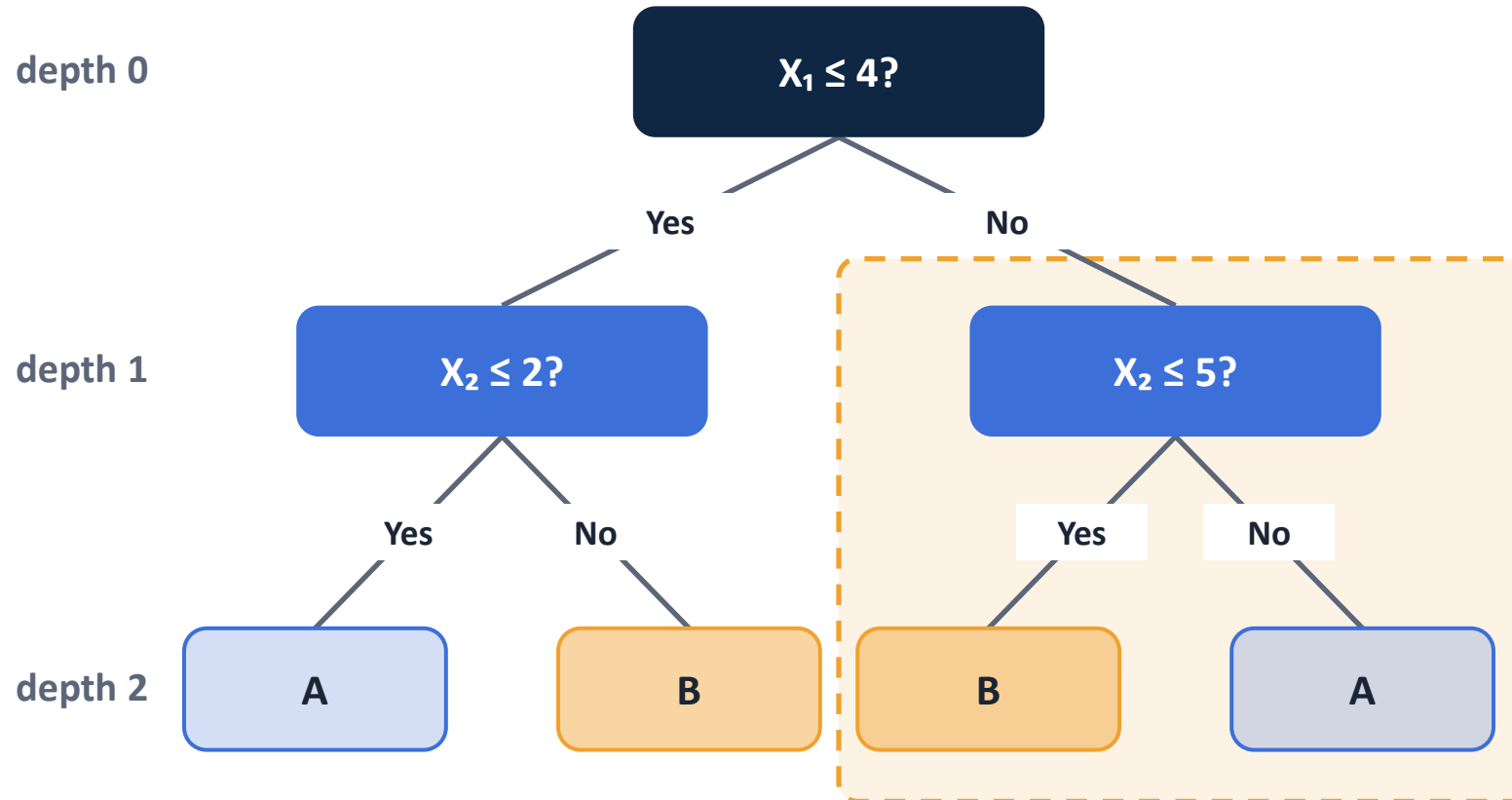
Leaf node

Final prediction



Anatomy of a decision tree

STRUCTURE



Root

first split, depth 0

Internal node

tests a feature

Branch

outcome of a test

Leaf

final prediction

Parent → child

dashed box: one parent, two children

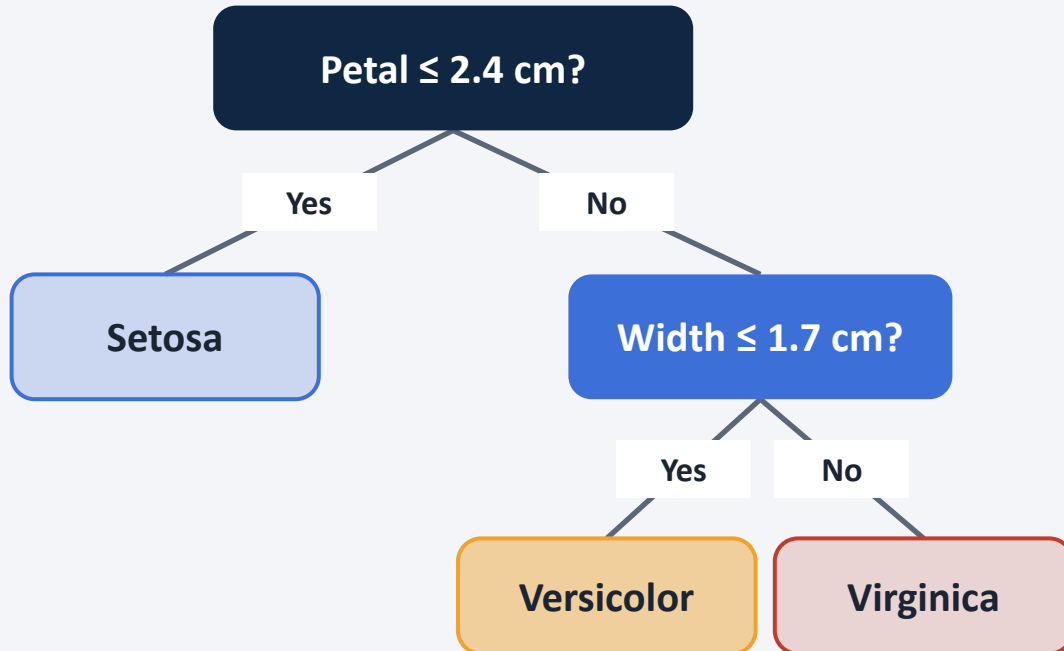
Depth = edges on the longest root-to-leaf path (here 2)

Classification vs regression

STRUCTURE

Classification

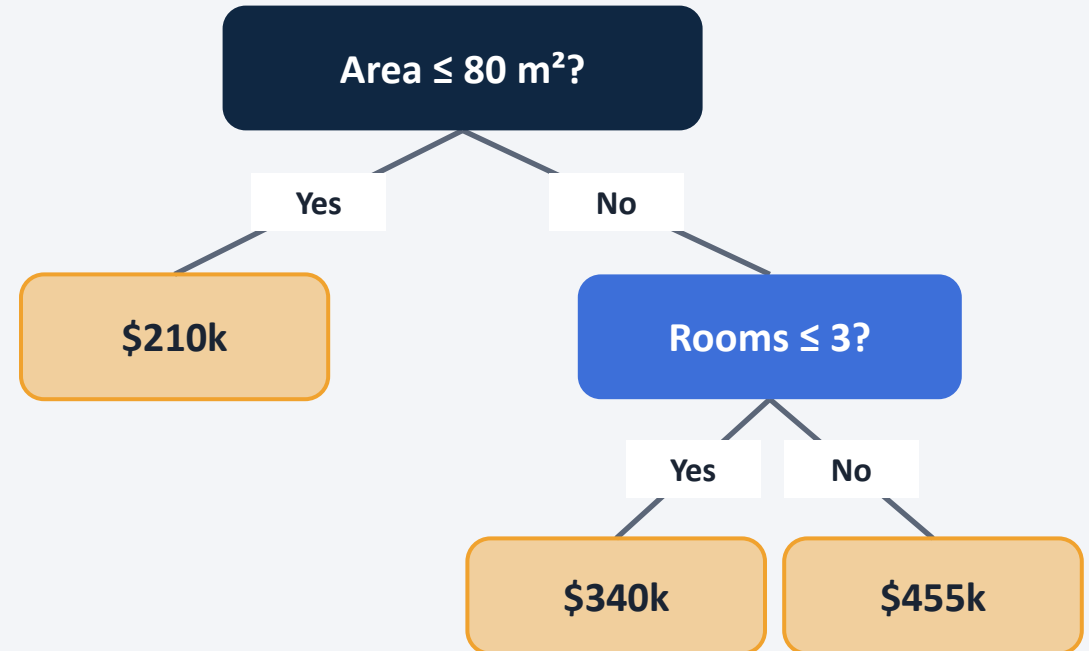
Output = class label



Leaf = majority class

Regression

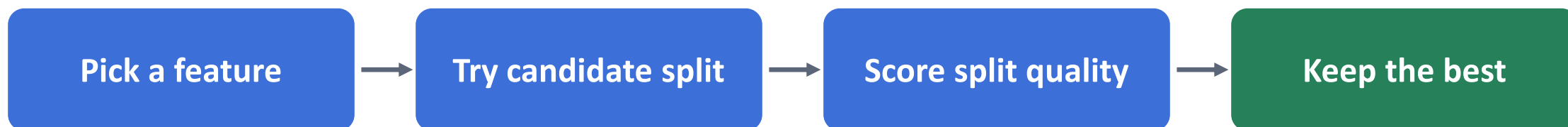
Output = continuous value



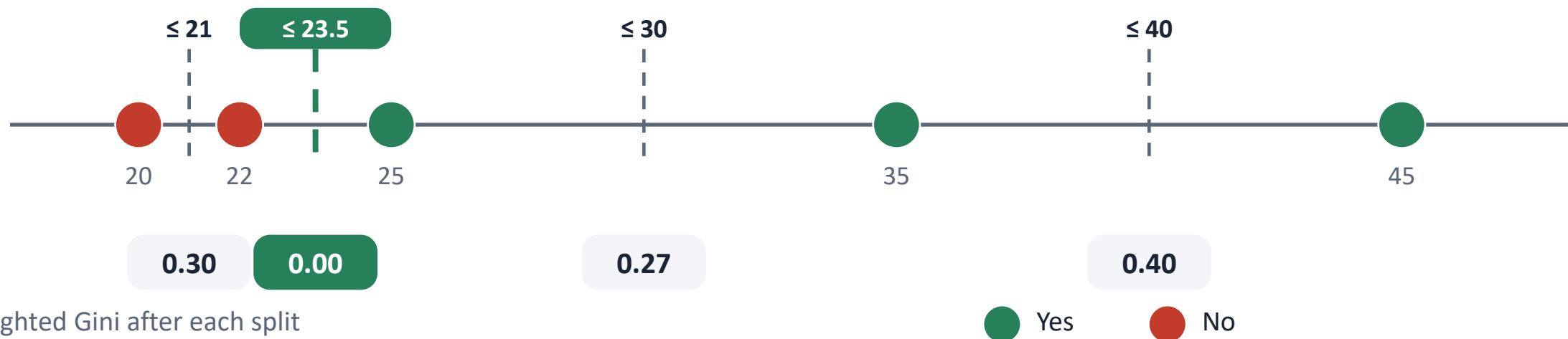
Leaf = mean of targets

How does a tree choose a split?

Which feature and threshold should I split on?

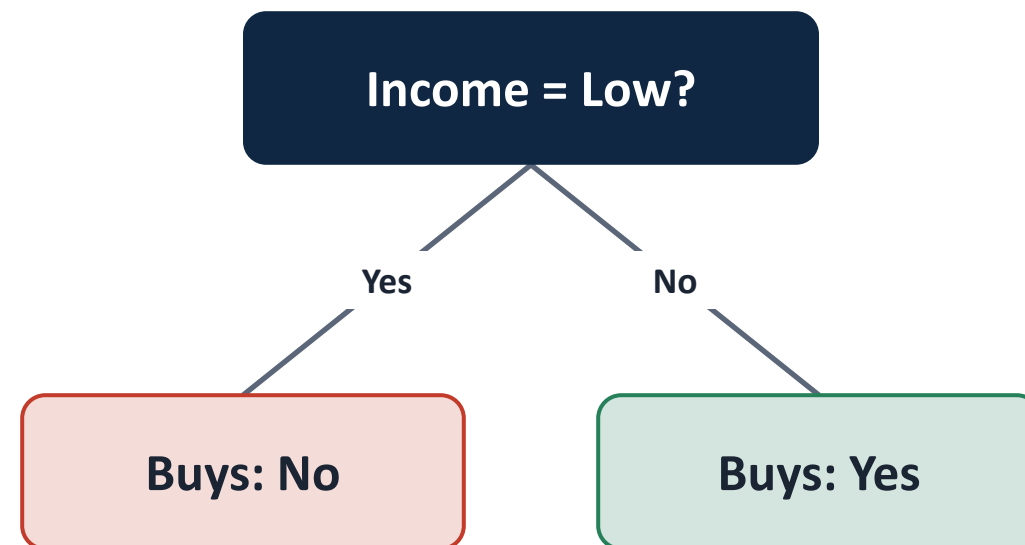


Feature: Age (sorted values, candidate thresholds at midpoints)



A small dataset

Age	Income	Student	Buys laptop
22	Low	Yes	No
25	Medium	Yes	Yes
35	High	No	Yes
45	Medium	No	Yes
20	Low	No	No
28	High	Yes	Yes
50	Low	Yes	No
38	Medium	Yes	Yes



One question separates all 8 rows

Pure vs impure nodes

SPLITTING

Pure node



$\text{Gini} = 0 \cdot \text{Entropy} = 0$

Impure node



$\text{Gini} = 0.5 \cdot \text{Entropy} = 1$

A good split makes the children purer than the parent

Parent



Left child



Right child



both pure

Gini impurity

SPLITTING

$$\text{Gini} = 1 - \sum p_i^2$$

p_i fraction of samples in class i

Σ sum over all classes

$$0 \leq \text{Gini} \leq 1 - 1/K$$

70 % Class A

30 % Class B

① Proportions

$$p_A = 0.7, p_B = 0.3$$

0.7 , 0.3

② Sum of squares

$$0.7^2 + 0.3^2 = 0.49 + 0.09$$

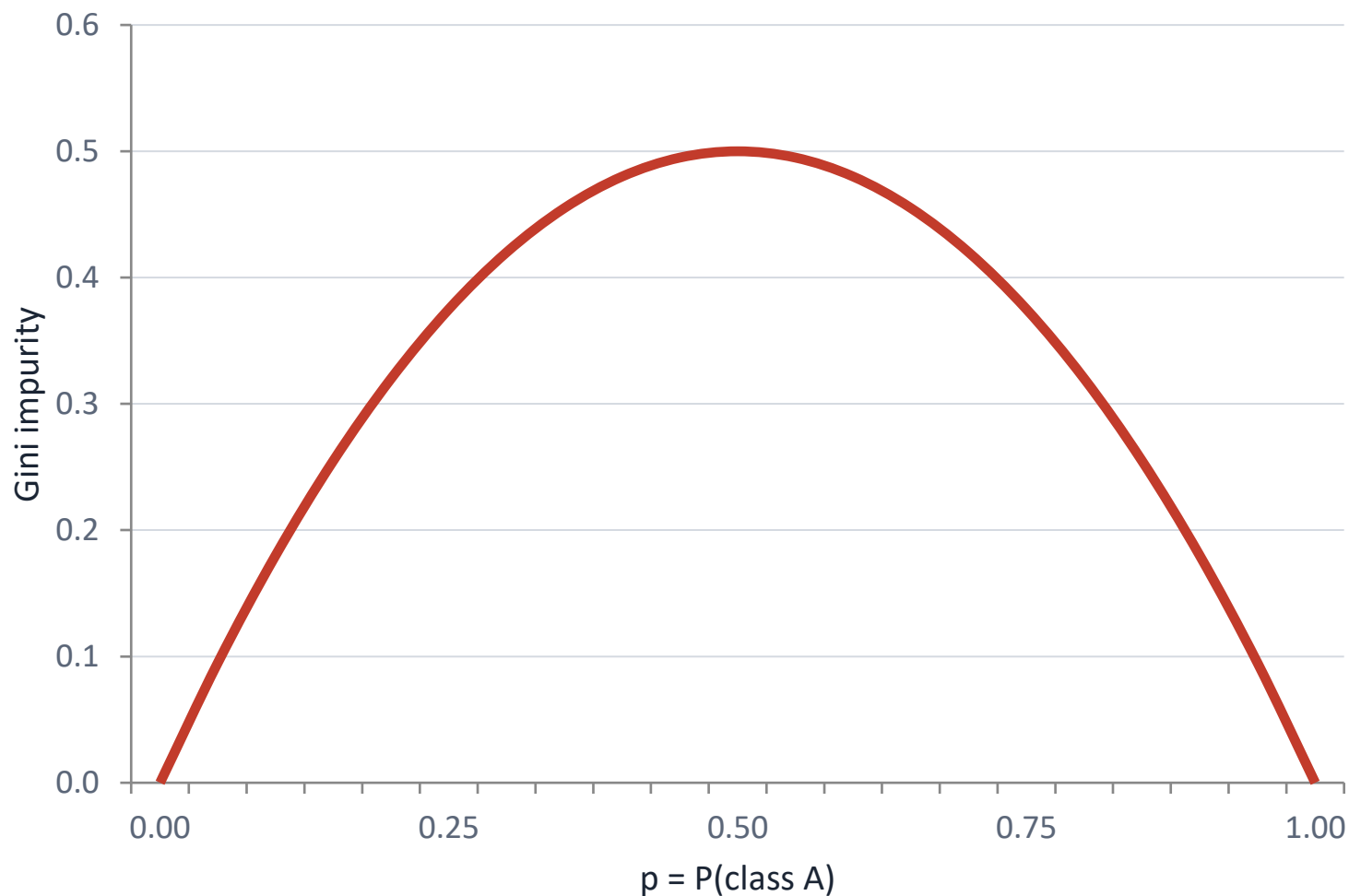
0.58

③ Subtract from 1

$$\text{Gini} = 1 - 0.58$$

0.42

Gini impurity vs class probability



$$\text{Gini} = 2p(1 - p)$$

$p = 0$ or 1

Gini = 0

pure node

$p = 0.5$

Gini = 0.5

maximum for 2 classes

$p = 0.7$

Gini = 0.42

our earlier example

Entropy

SPLITTING

$$H(S) = - \sum p_i \log_2 p_i$$

p_i class proportion in S
 $\log_2$ result measured in bits
 $0 \cdot \log_2 0 := 0$

LOW ENTROPY: ordered, pure

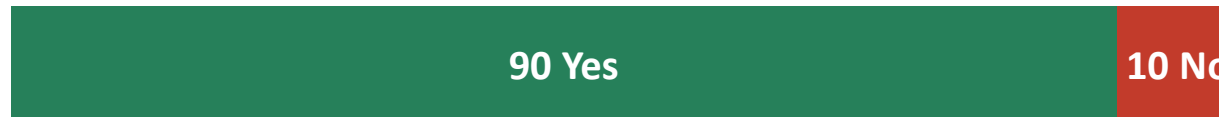
HIGH ENTROPY: mixed, uncertain

100 / 0



$H = 0$

90 / 10



$H = 0.469$

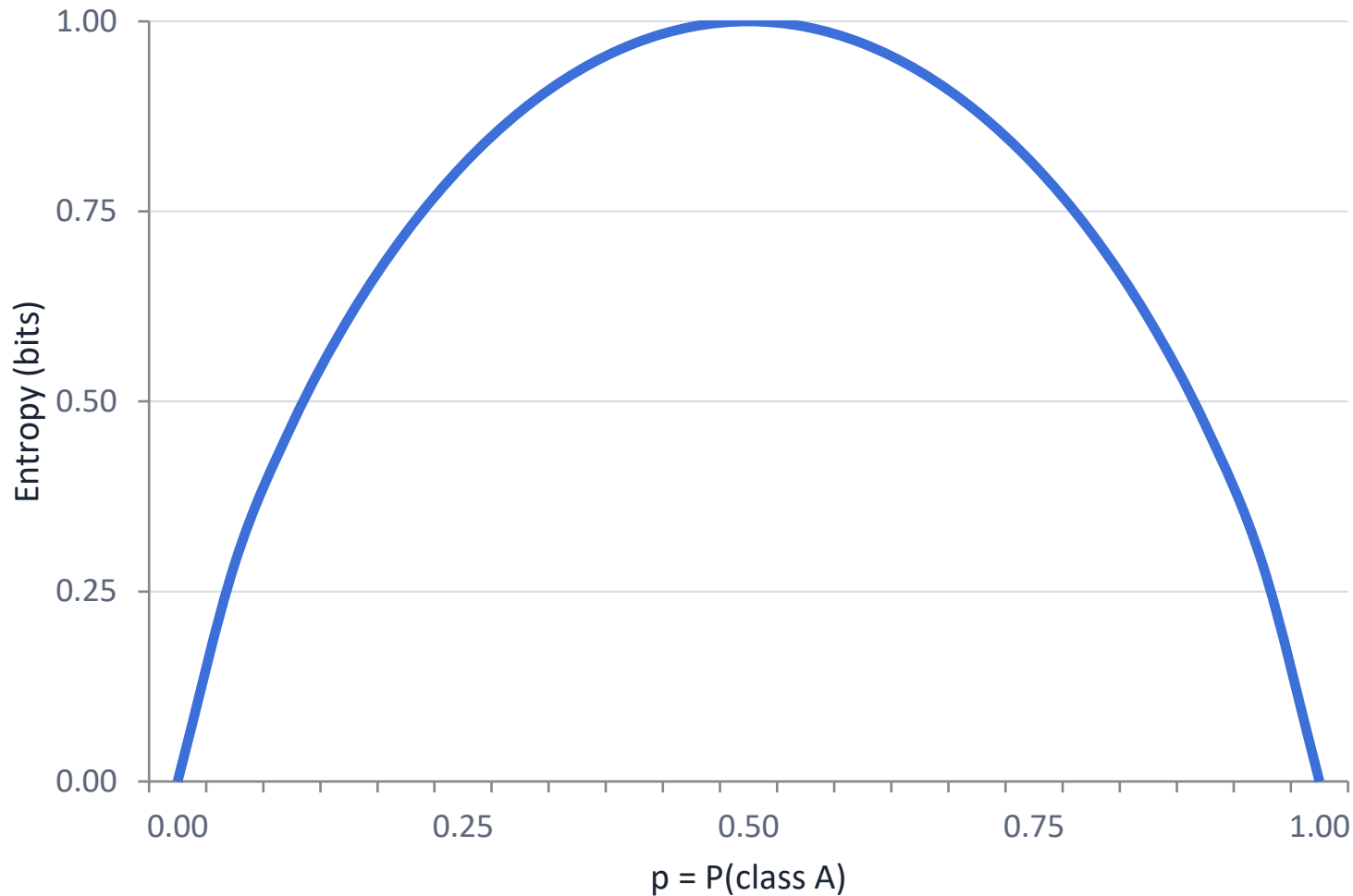
50 / 50



$H = 1.000$

Entropy vs class probability

SPLITTING



$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$$

$H = 0$

pure: $p = 0$ or 1

$H = 1$

max uncertainty: $p = 0.5$

$H(0.9) = 0.469$

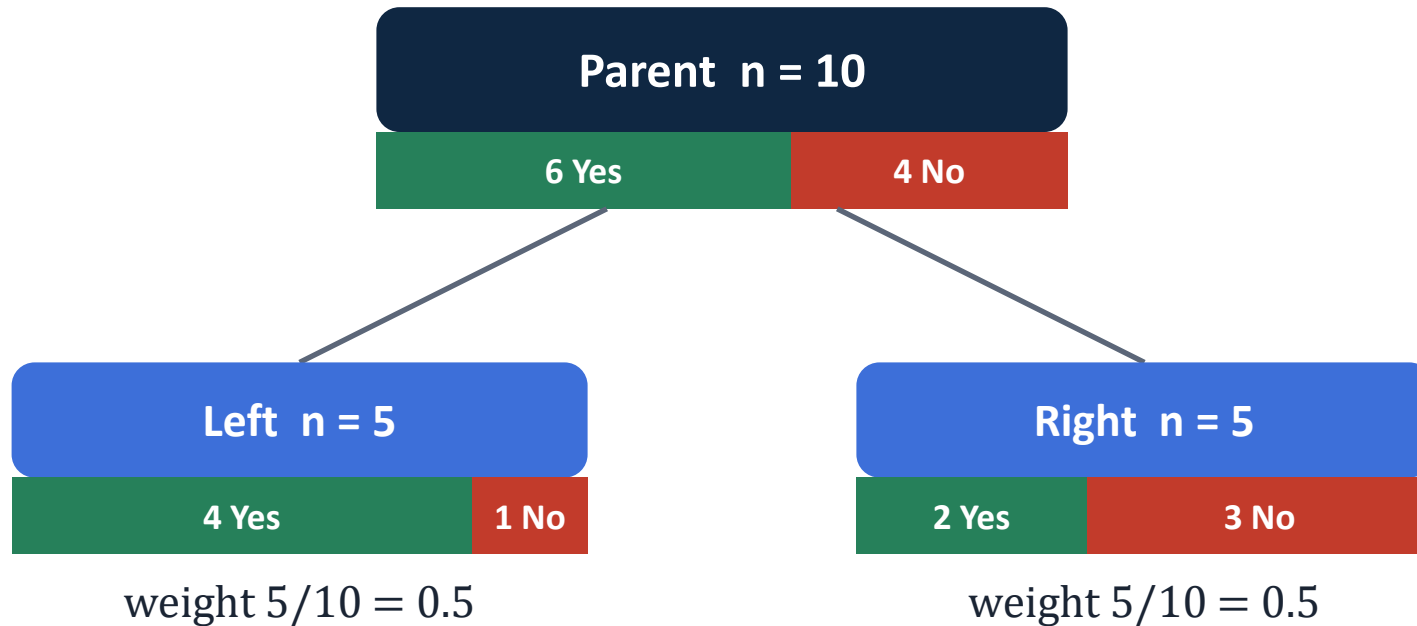
mostly pure

Information gain

SPLITTING

$$\text{Gain} = I(\text{parent}) - \sum_k (n_k / n) \cdot I(\text{child}_k)$$

$I(\cdot)$ = Gini or entropy



Impurity

a property of one node

Gain

a property of a split: impurity reduction

Weights stop tiny children from dominating

Information gain: worked example

SPLITTING

Parent: 6 Yes / 4 No



Left: 4 Yes / 1 No

Right: 2 Yes / 3 No

① Parent entropy

$$-(0.6 \log_2 0.6 + 0.4 \log_2 0.4)$$

$$H = 0.971$$

② Child entropies

$$\begin{aligned} H(L) &= -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) \\ H(R) &= -(0.4 \log_2 0.4 + 0.6 \log_2 0.6) \end{aligned}$$

$$0.722 \mid 0.971$$

③ Weighted child

$$\begin{aligned} &0.5 \times 0.722 \\ &+ 0.5 \times 0.971 \end{aligned}$$

$$0.846$$

④ Information gain

$$0.971 - 0.846$$

$$0.125 \text{ bits}$$

Gini vs entropy

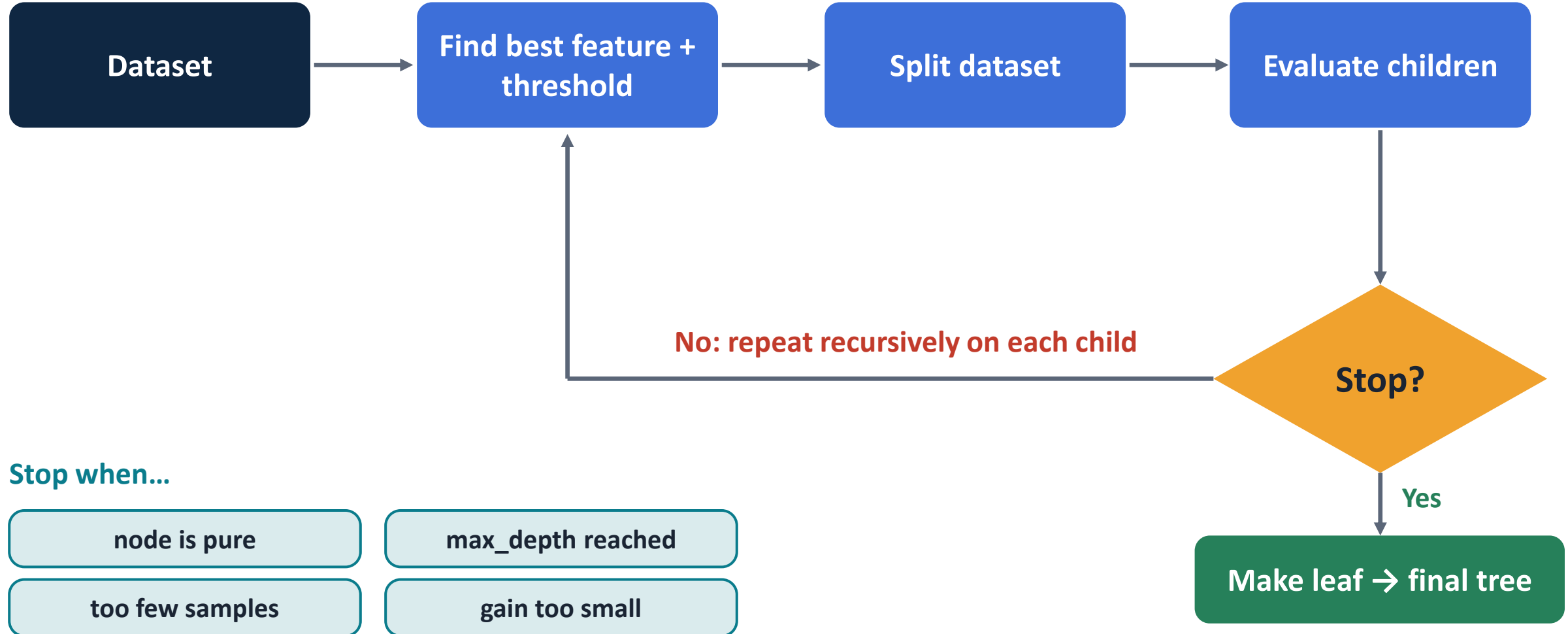
SPLITTING

Criterion	Gini	Entropy
Formula	$1 - \sum p_i^2$	$-\sum p_i \log_2 p_i$
Interpretation	Chance of mislabelling a random sample	Average uncertainty, in bits
Range (2 classes)	0 to 0.5	0 to 1
Range (K classes)	0 to $1 - 1/K$	0 to $\log_2 K$
Computation	Cheaper: no logarithm	Slightly costlier: logarithm
Typical usage	CART; scikit-learn default	ID3 / C4.5; criterion="entropy"

Both generally produce very similar trees

How the tree is built

ALGORITHM



Greedy splitting

ALGORITHM

At each node: take the best split now

Income = High



0.213

Age ≤ 30



0.080

Student = Yes



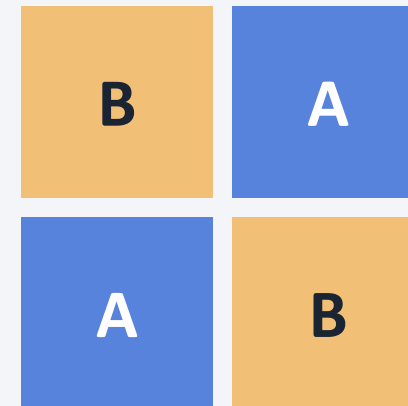
0.013

impurity decrease (Gini)

Pick the max → commit → recurse. No backtracking.

It does NOT search every possible tree

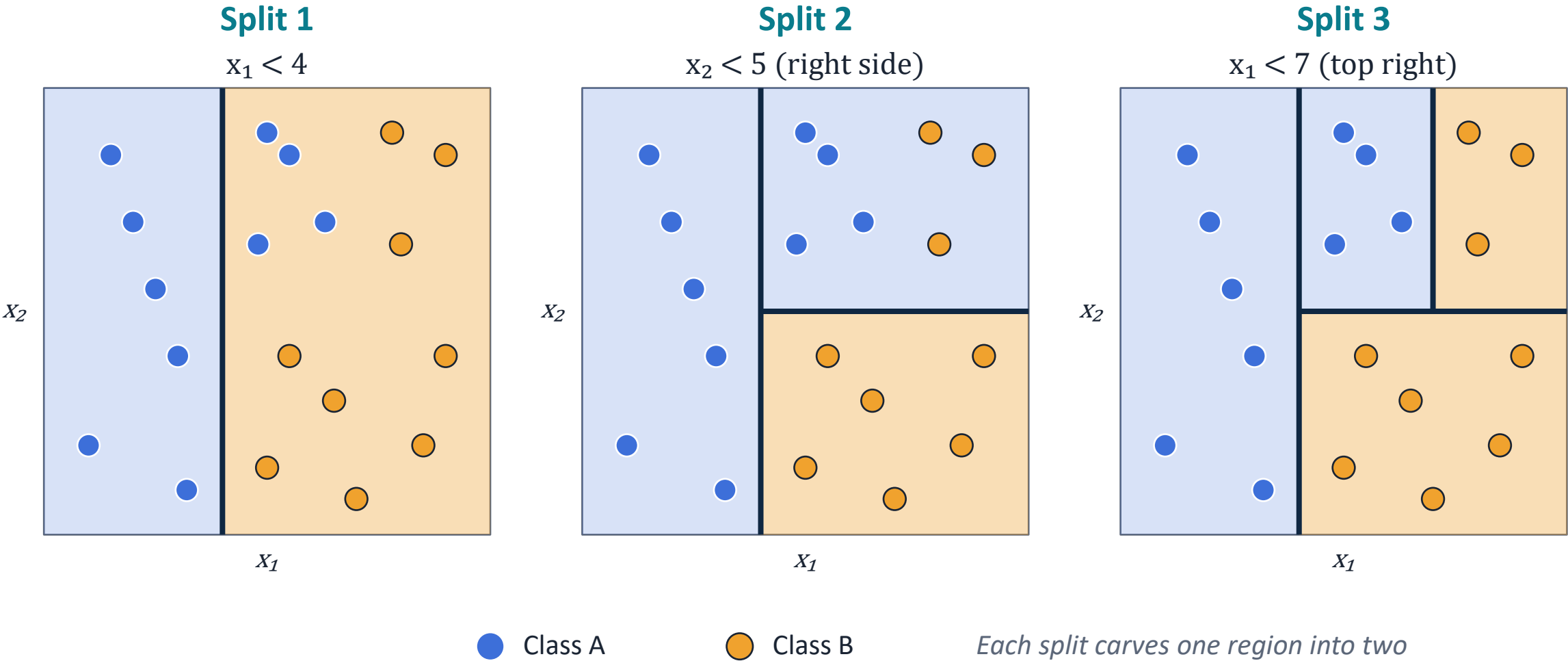
Locally best $\neq$ globally best



XOR: a single split has ~ 0 gain, yet two splits are perfect

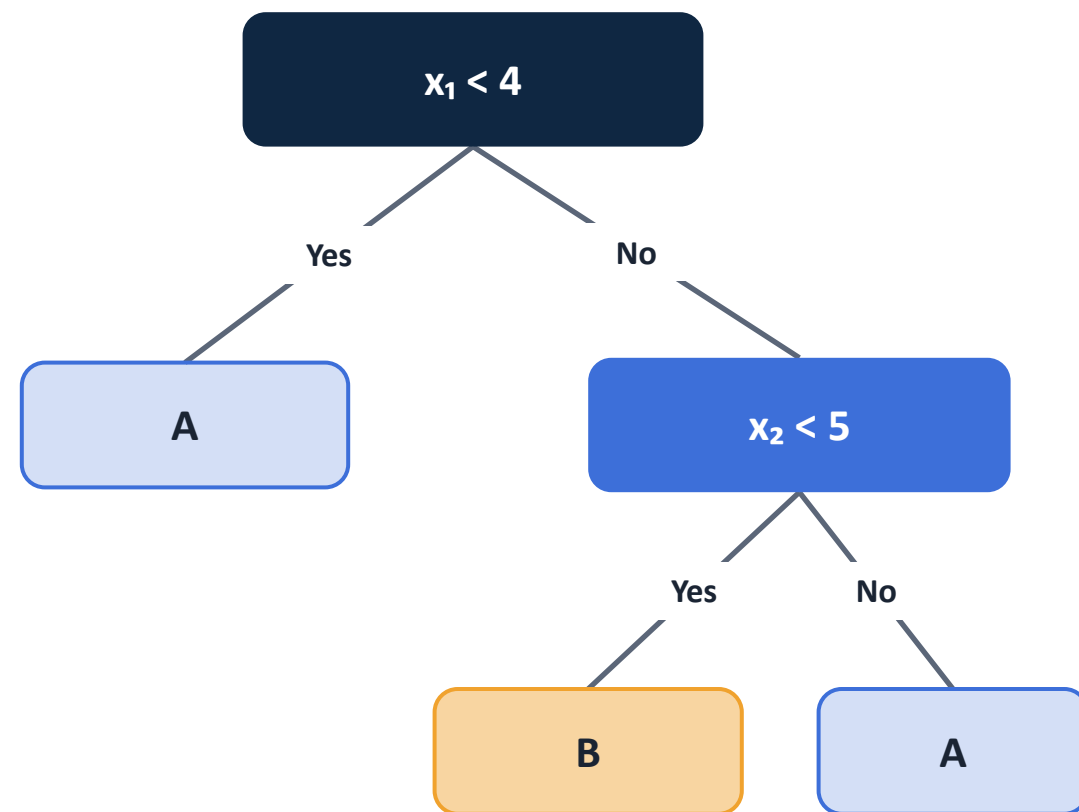
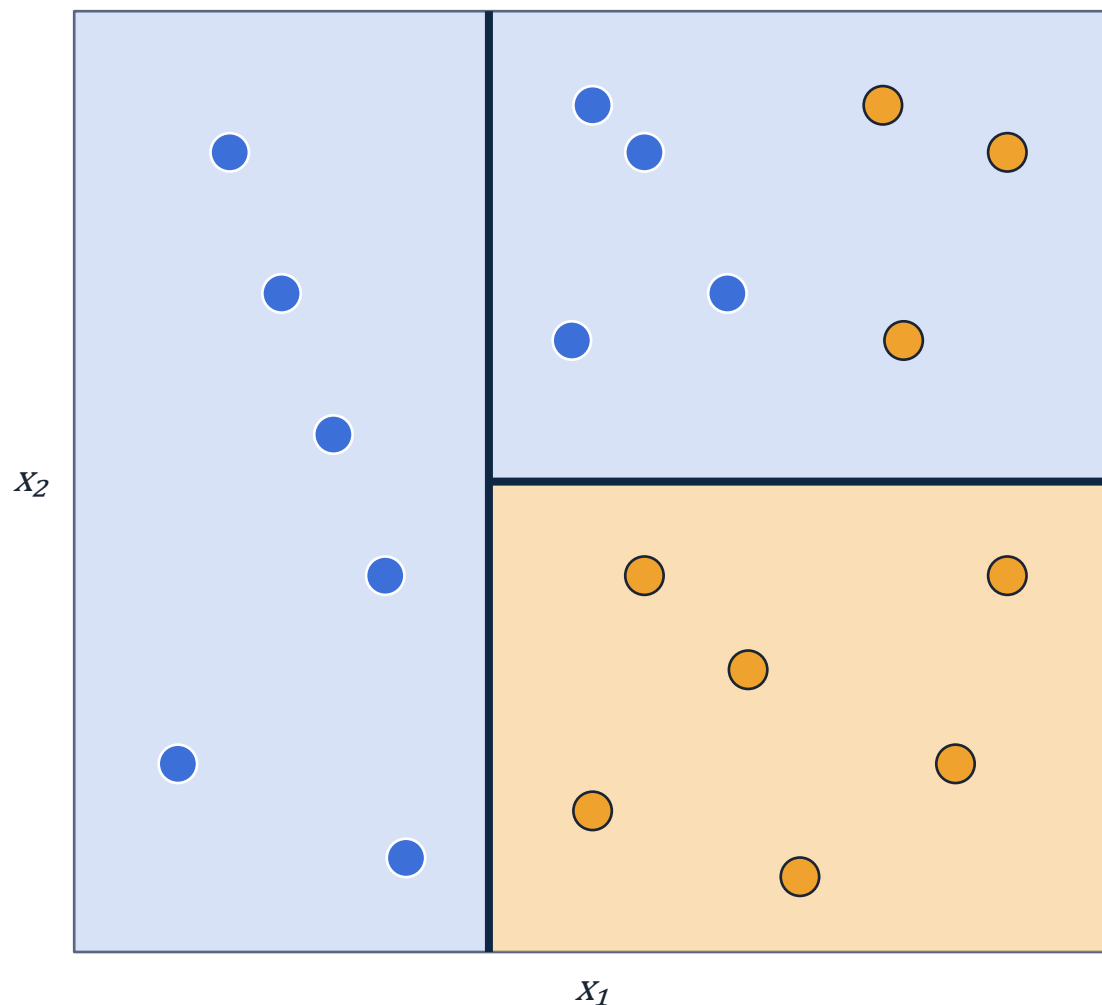
Recursive partitioning of feature space

ALGORITHM



Axis-aligned decision boundaries

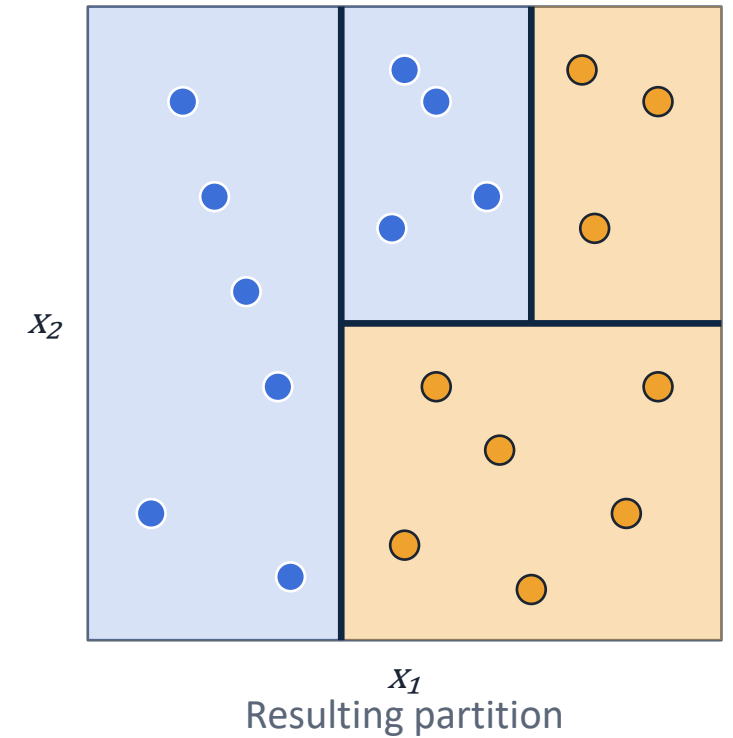
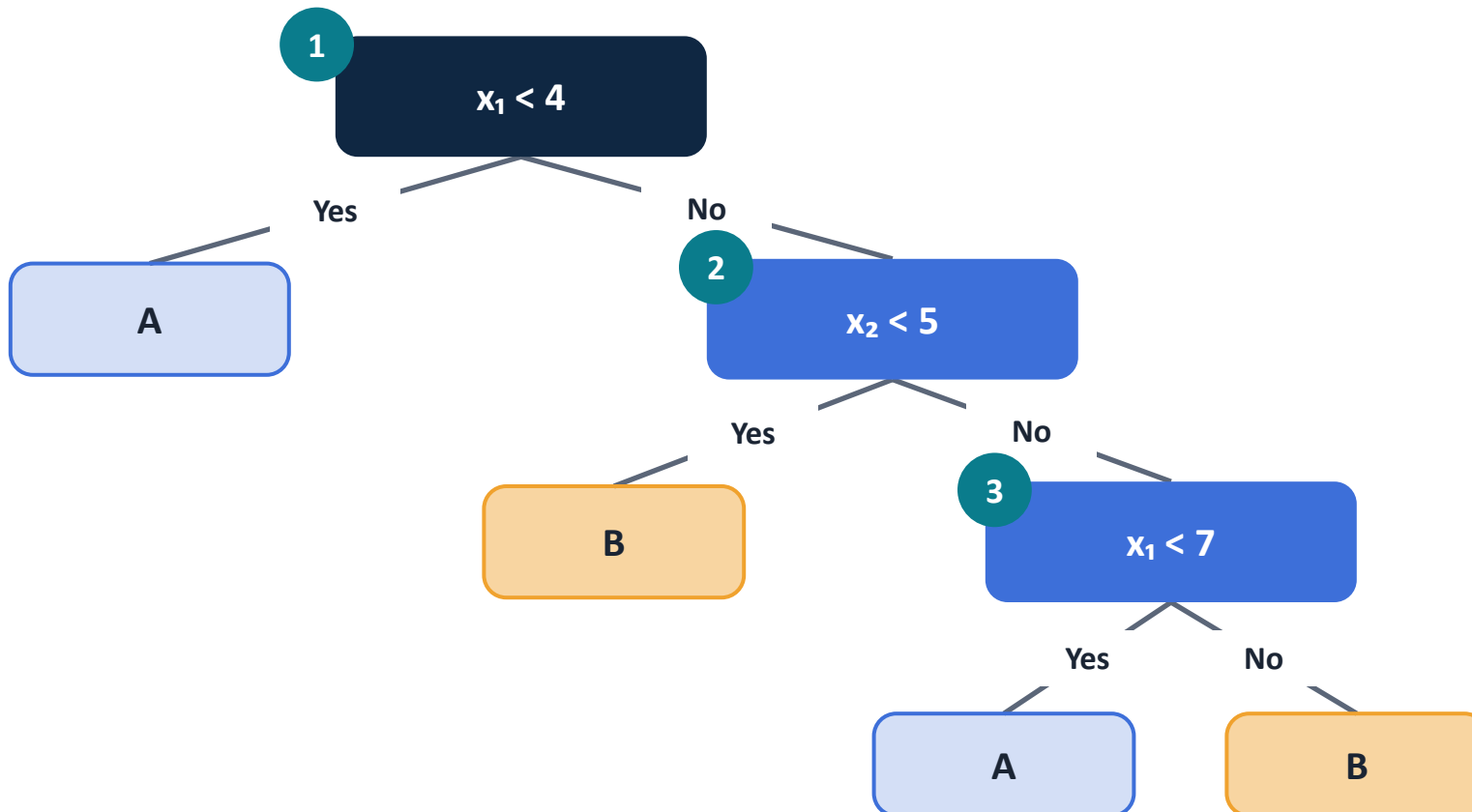
ALGORITHM



Yes: $x_1 < 4$ · No: $x_1 \geq 4$ · boundaries are parallel to the axes

Building the tree from data

ALGORITHM



Stopping conditions

OVERFITTING

1 `max_depth`

Cap the number of levels

2 `min_samples_split`

Samples needed to split a node

3 `min_samples_leaf`

Samples every leaf must keep

4 `max_leaf_nodes`

Cap the total number of leaves

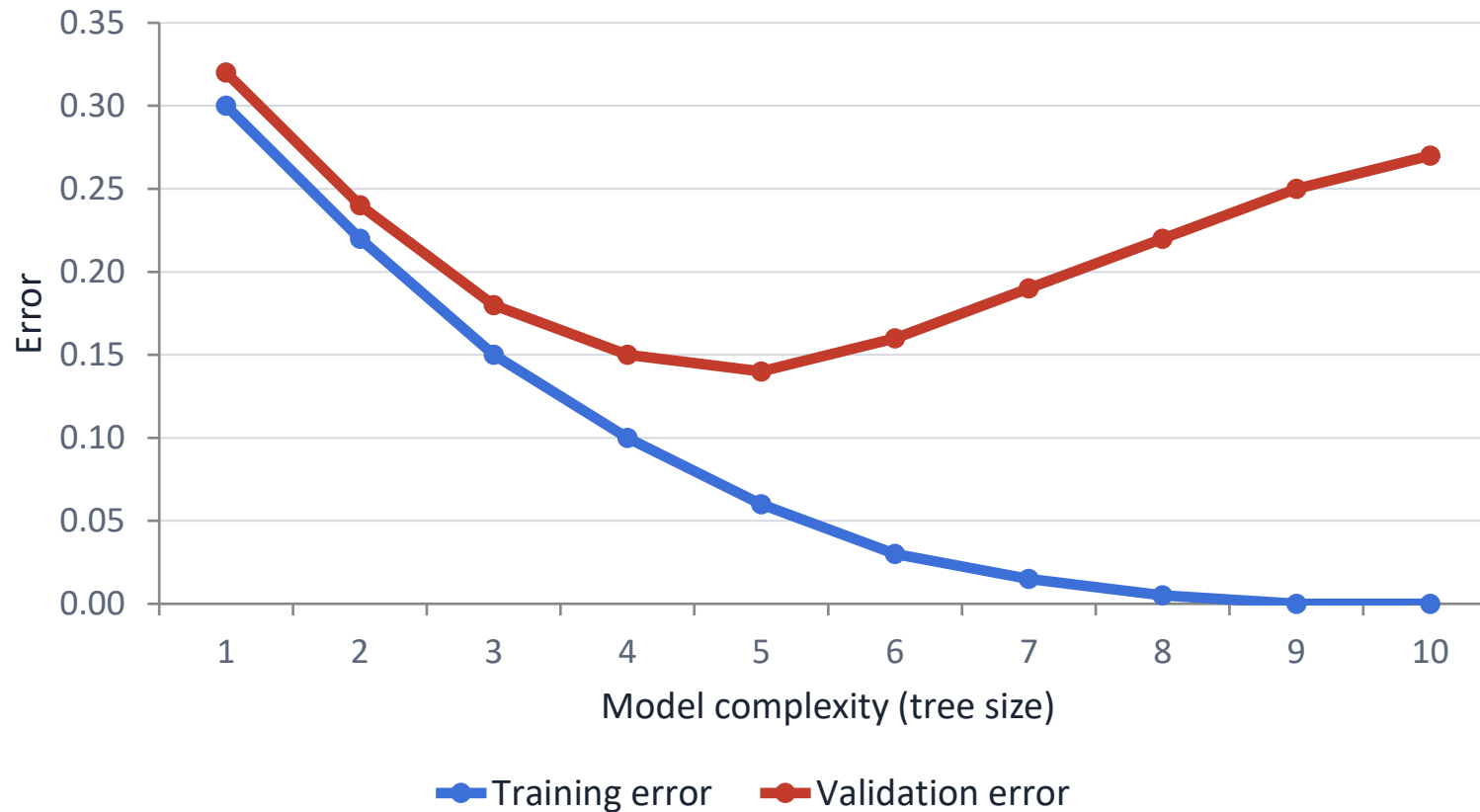
5 `min_impurity_decrease`

Split only if impurity drops enough

Stopping early → simpler trees, less memorisation of noise

Underfitting → good fit → overfitting

OVERFITTING



Underfitting

both errors high

Good fit

validation error lowest

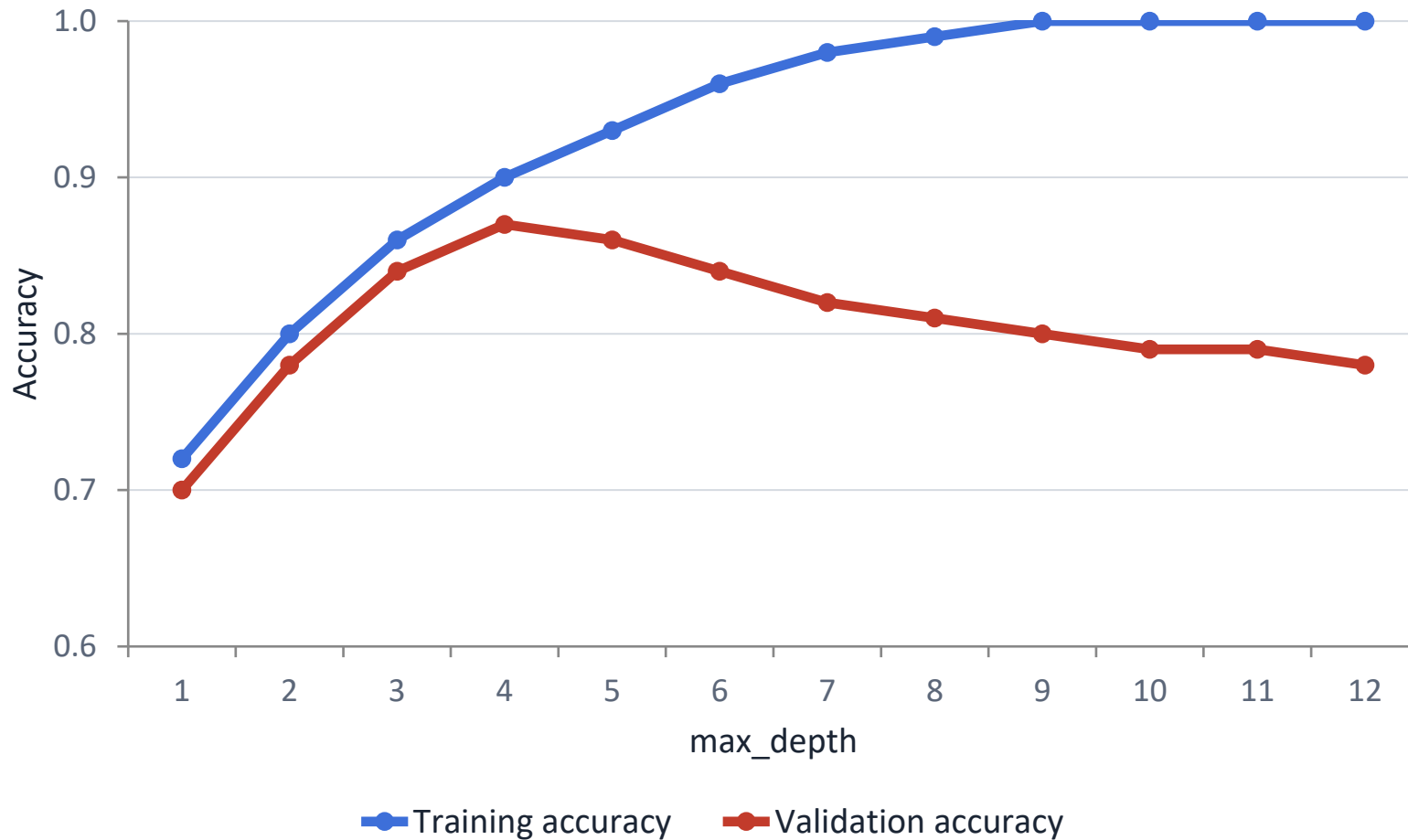
Overfitting

train $\approx$ 0, validation rising

An unrestricted tree splits until every leaf is pure: it memorises the training set, noise included

Tree depth vs performance

OVERFITTING



Best depth ≈ 4

validation accuracy peaks

Gap widens after

overfitting begins

Choose by validation

never by training score

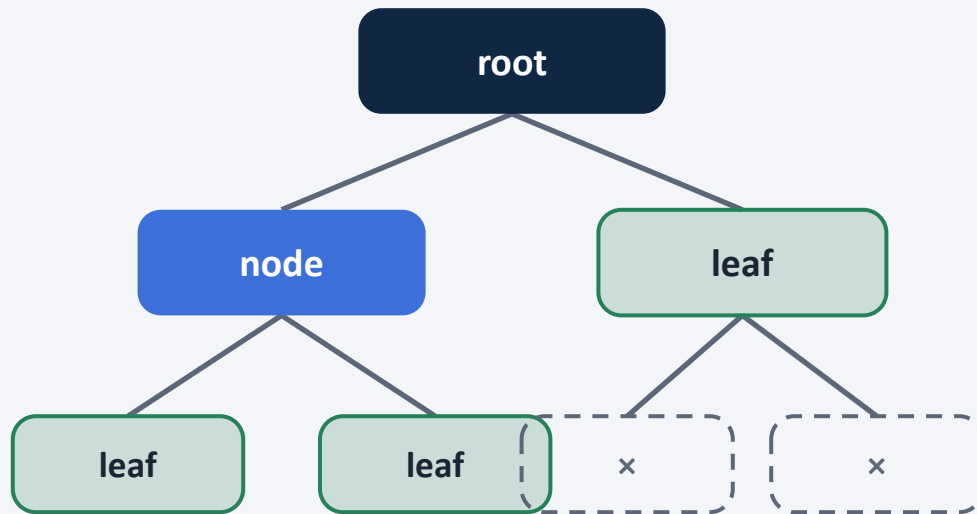
Illustrative curves

Pruning

OVERFITTING

Pre-pruning

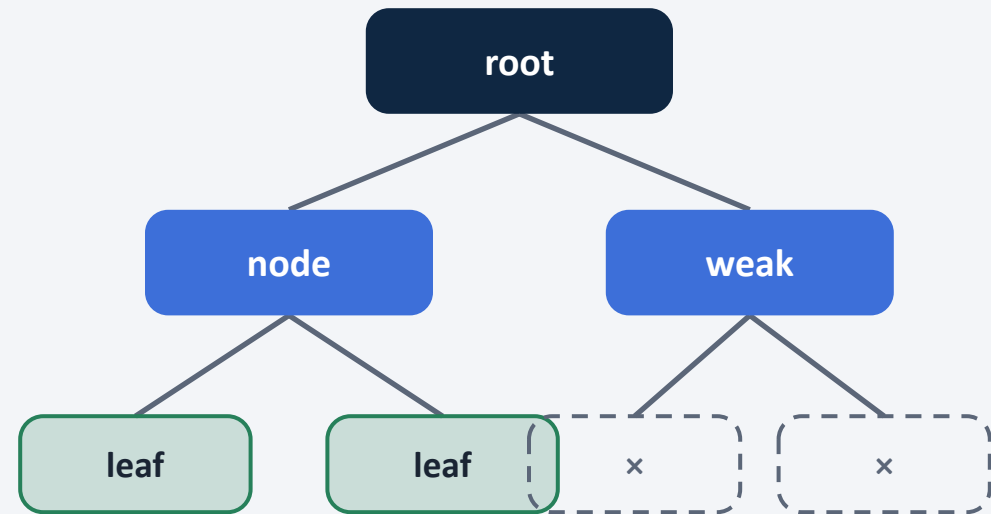
Stop growing early



`max_depth · min_samples_leaf ...`

Post-pruning

Grow, then cut weak branches



$R_{\alpha}(T) = R(T) + \alpha \cdot |T|$ `ccp_alpha`

Reduce complexity

Improve generalisation

Improve interpretability

Advantages

TRADE-OFFS



Easy to interpret

Rules you can read aloud



No feature scaling

Splits depend on order, not magnitude



Nonlinear patterns

Captures interactions naturally



Mixed feature types

Categoricals need encoding in scikit-learn



Feature selection aid

Splits hint at useful features



Easy to visualise

`plot_tree` and `export_text`

Limitations

TRADE-OFFS



Can overfit

Memorises noise if unrestricted



High variance

Predictions depend heavily on the sample



Unstable

Small data changes can give different trees



Greedy, not global

Best-now splits may be suboptimal



Axis-aligned only

Diagonal boundaries need many steps



Can grow huge

Deep trees lose interpretability

Decision tree vs other models

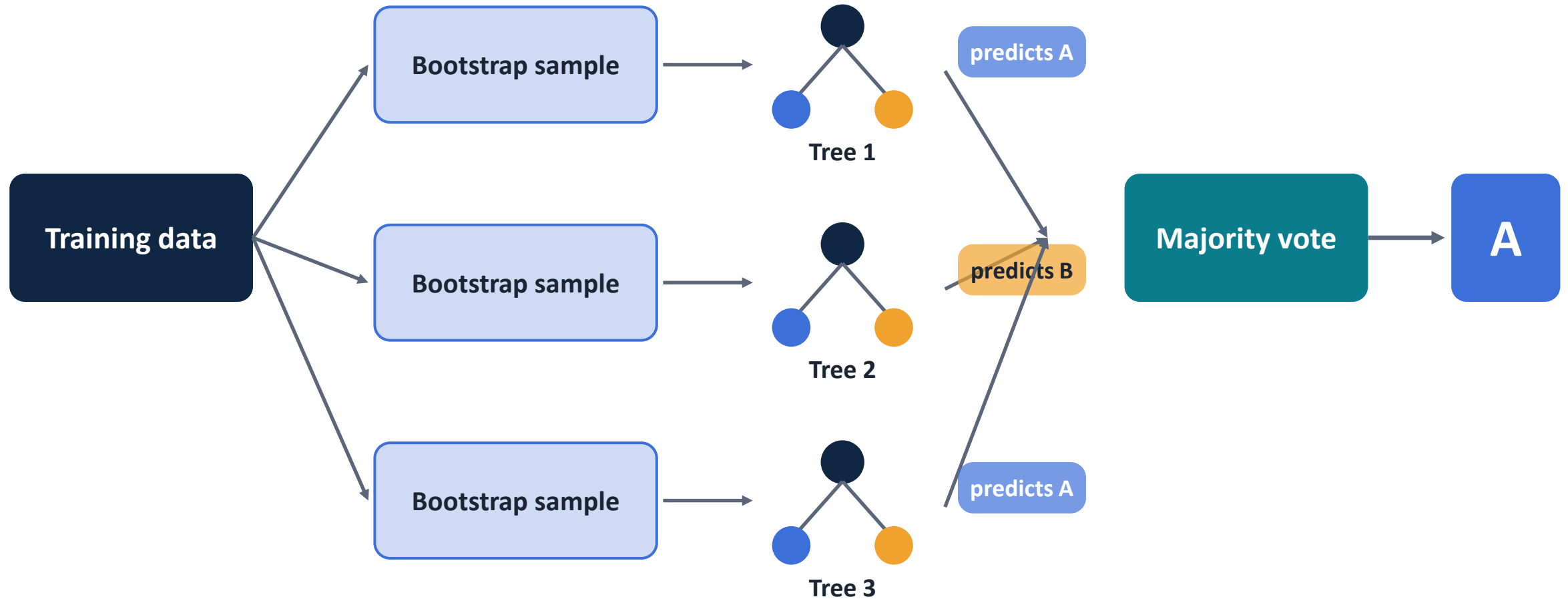
TRADE-OFFS

Criterion	Decision tree	Logistic reg.	KNN	Random forest
Interpretability	High	High (coefficients)	Low	Low (ensemble)
Nonlinearity	Yes, axis-aligned	Linear boundary	Yes, local	Yes
Preprocessing	No scaling; encode categoricals	Scale and encode	Scaling essential	No scaling; encode categoricals
Overfitting	High variance	Lower variance	Depends on k	Reduced by averaging
Training speed	Fast	Fast	No training; slow predictions	Slower; parallelisable

Typical tendencies, not guarantees

Random forest connection

TRADE-OFFS



Averaging B decorrelated trees: $\text{Var} \approx \sigma^2 / B$

Python implementation

IMPLEMENTATION

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

criterion

impurity measure

max_depth

limit tree levels

min_samples_*

minimum samples to split / per leaf

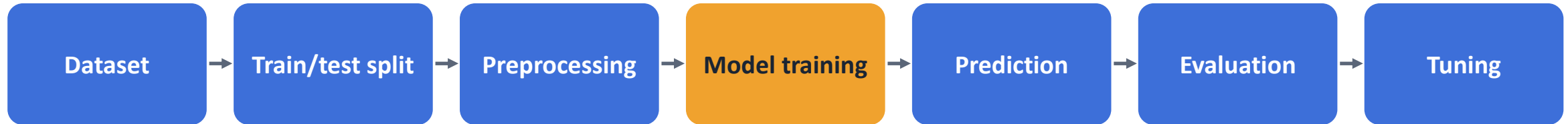
random_state

reproducible results

No feature scaling needed · encode categorical columns first

A complete ML pipeline

IMPLEMENTATION



Decision tree fits here ↑

```
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Evaluate with

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix

Visualising the tree

IMPLEMENTATION

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(14, 7))
plot_tree(
    model,
    feature_names=feature_names,
    class_names=class_names,
    filled=True,
    rounded=True,
)
plt.show()
```

Reading one node

feature $\leq$ threshold
gini = 0.468
samples = 455
value = [170, 285]
class = benign

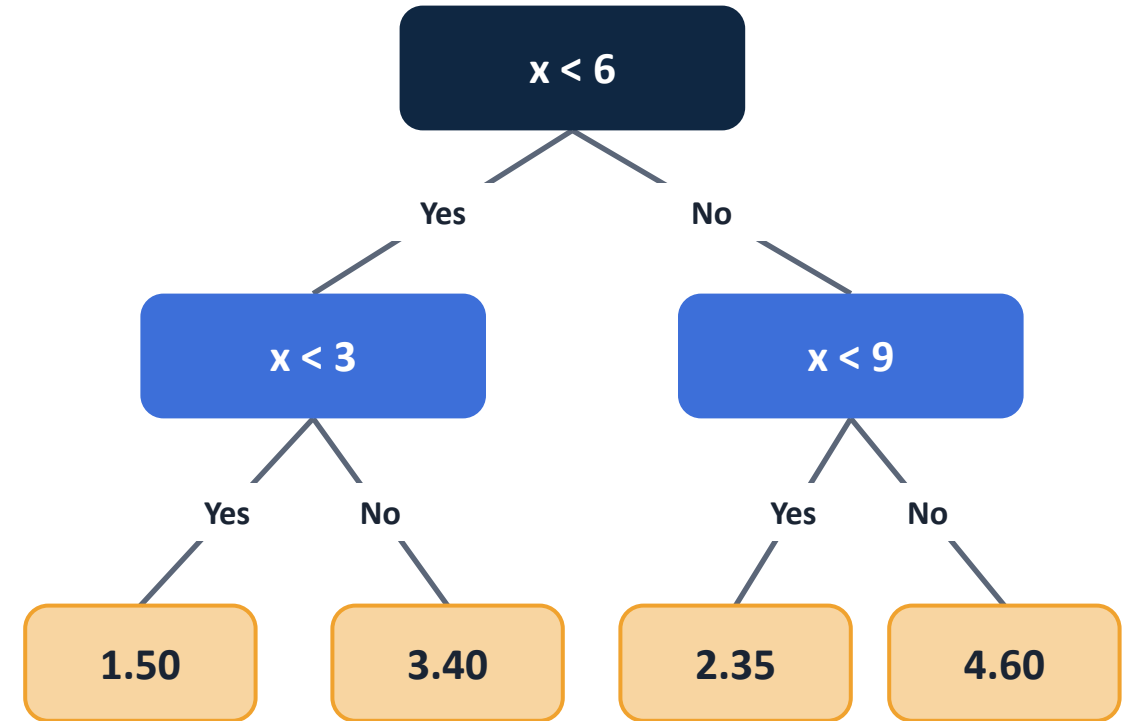
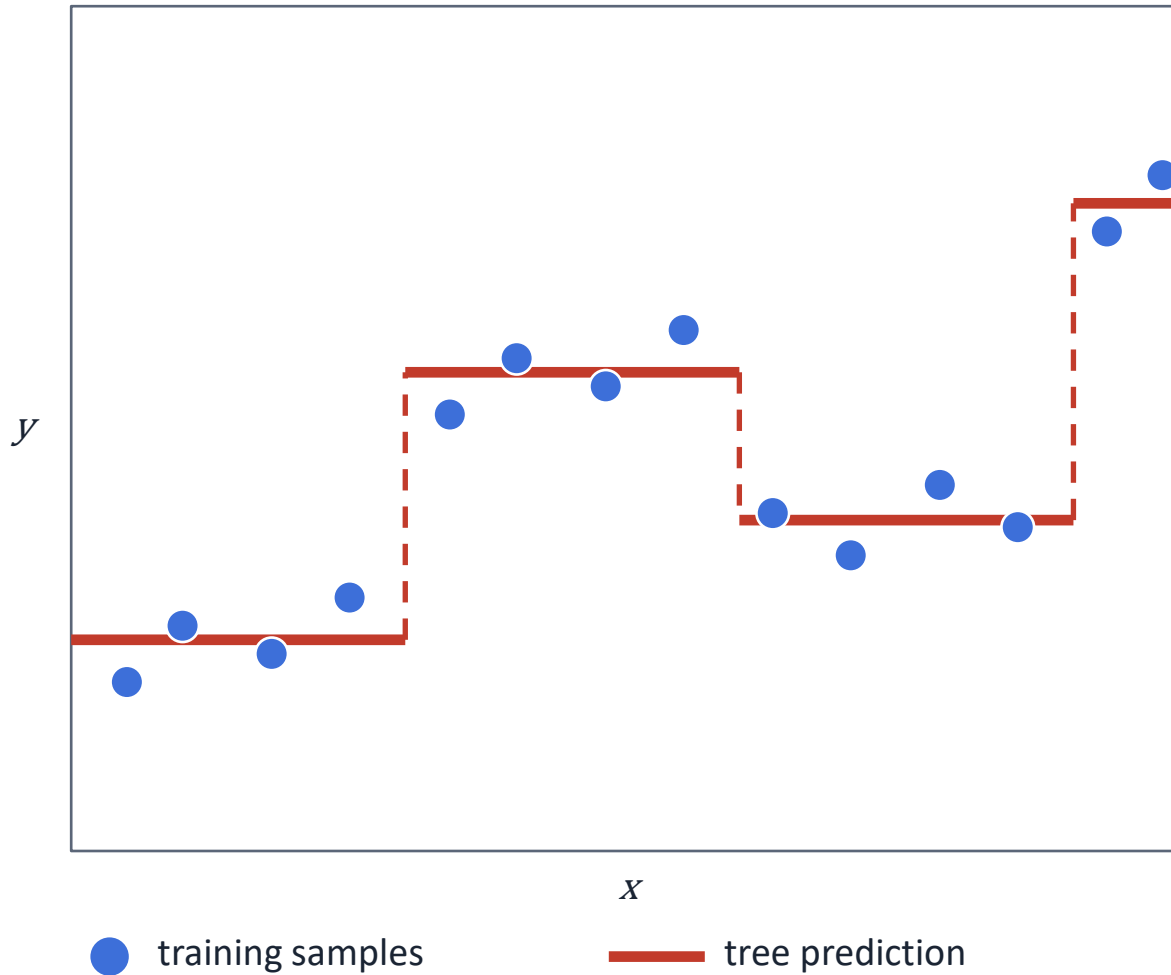
- the test (Yes → left)
- node impurity
- samples reaching node
- class counts
- majority class

Fill colour = majority class; darker = purer node

Illustrative values

Decision tree regression

REGRESSION

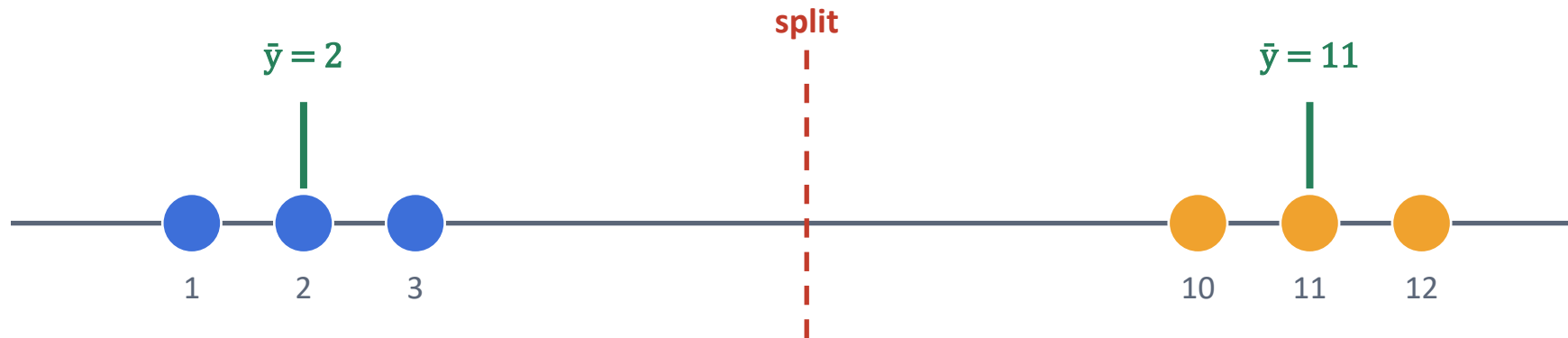


Leaf value = mean of training targets in that region

Regression splitting: error reduction

$$\text{MSE}(S) = (1/n) \sum (y_i - \bar{y})^2$$

$$\text{Reduction} = \text{MSE}(\text{parent}) - \sum_k (n_k/n) \cdot \text{MSE}(\text{child}_k)$$



Parent MSE

20.92

Weighted child MSE

0.67

Reduction

20.25

$y = 1, 2, 3, 10, 11, 12 \rightarrow$ after the split, each side is nearly constant

Key hyperparameters cheat sheet

PRACTICE

criterion

Split-quality measure

`gini · entropy · log_loss`

max_depth

Caps tree levels

smaller → simpler, may underfit

min_samples_split

Samples needed to split

larger → simpler

min_samples_leaf

Samples kept per leaf

larger → smoother, stabler

max_leaf_nodes

Caps total leaves

smaller → simpler

ccp_alpha

Cost-complexity pruning

larger → smaller tree

random_state

Fixes randomness

reproducible results

Hyperparameter tuning

PRACTICE

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "max_depth": [2, 3, 4, 5, 6, 8, 10],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
}
search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid, cv=5, scoring="f1",
)
search.fit(X_train, y_train)
print(search.best_params_)
```

5-fold cross-validation

F1	val	train	train	train	train
F2	train	val	train	train	train
F3	train	train	val	train	train
F4	train	train	train	val	train
F5	train	train	train	train	val

Average the 5 validation scores

$7 \times 3 \times 3 = 63$ settings $\times$ 5 folds = 315 fits · RandomizedSearchCV samples `n_iter` settings instead

Common mistakes

PRACTICE



Unlimited depth

Set `max_depth` or `min_samples_leaf`



Only training accuracy

Always evaluate on held-out data



Ignoring validation scores

Tune on validation, report on test



Assuming deeper = better

Check the validation curve



No cross-validation

Use `cv` for stable estimates



Misreading feature importance

Impurity-based $\neq$ causal; try permutation importance



Not understanding the data

Explore, clean, check for leakage first

Golden rule: a training score alone proves nothing

Real-world applications

PRACTICE



Loan approval

Transparent eligibility rules



Medical decision support

Rule-like aids, used with clinicians



Fraud detection

Flag unusual transactions



Customer churn

Who is likely to leave?



Credit risk

Estimate default risk



Manufacturing QC

Pass / fail from sensor readings



Marketing segmentation

Interpretable customer groups

Mini activity: which split first?

#	Age ≤ 30	Income high	Student	Buys
1	Yes	No	Yes	No
2	Yes	No	No	No
3	Yes	Yes	No	Yes
4	Yes	Yes	Yes	Yes
5	No	Yes	No	Yes
6	No	No	No	Yes
7	No	Yes	Yes	Yes
8	No	No	Yes	No
9	No	No	No	Yes
10	Yes	No	No	No

Parent: 6 Yes / 4 No

Split A Age ≤ 30 ?

Split B Income high?

Split C Student?

**Compute the weighted Gini of each split.
Which is lowest?**

Answer and discussion

REVIEW

Split	Group 1: n (Yes/No) · Gini	Group 2: n (Yes/No) · Gini	Weighted	Decrease
Income high?	High: 4 (4/0) · 0.000	Low: 6 (2/4) · 0.444	0.267	0.213
Age ≤ 30?	Yes: 5 (2/3) · 0.480	No: 5 (4/1) · 0.320	0.400	0.080
Student?	Yes: 4 (2/2) · 0.500	No: 6 (4/2) · 0.444	0.467	0.013

Winner: Income high?

$$\text{Parent Gini} = 1 - (0.6^2 + 0.4^2) = 0.480$$

$$\text{Weighted} = 0.4 \times 0 + 0.6 \times 0.444 = 0.267$$

$$\text{Decrease} = 0.480 - 0.267 = 0.213$$

Interview and exam questions

REVIEW

1 What is a decision tree?

2 What is Gini impurity?

3 What is entropy?

4 What is information gain?

5 Why do decision trees overfit?

6 How does max_depth affect the model?

7 Gini vs entropy?

8 Classification vs regression tree?

9 Why are trees non-parametric?

10 Why are random forests more robust?

Key takeaways

REVIEW

1

A tree predicts by asking feature–threshold questions

2

Best split = largest impurity reduction (Gini or entropy)

3

Information gain uses weighted child impurity

4

Building is greedy and recursive

5

Unrestricted trees overfit: limit depth, prune

6

No scaling needed; encode categoricals in scikit-learn

7

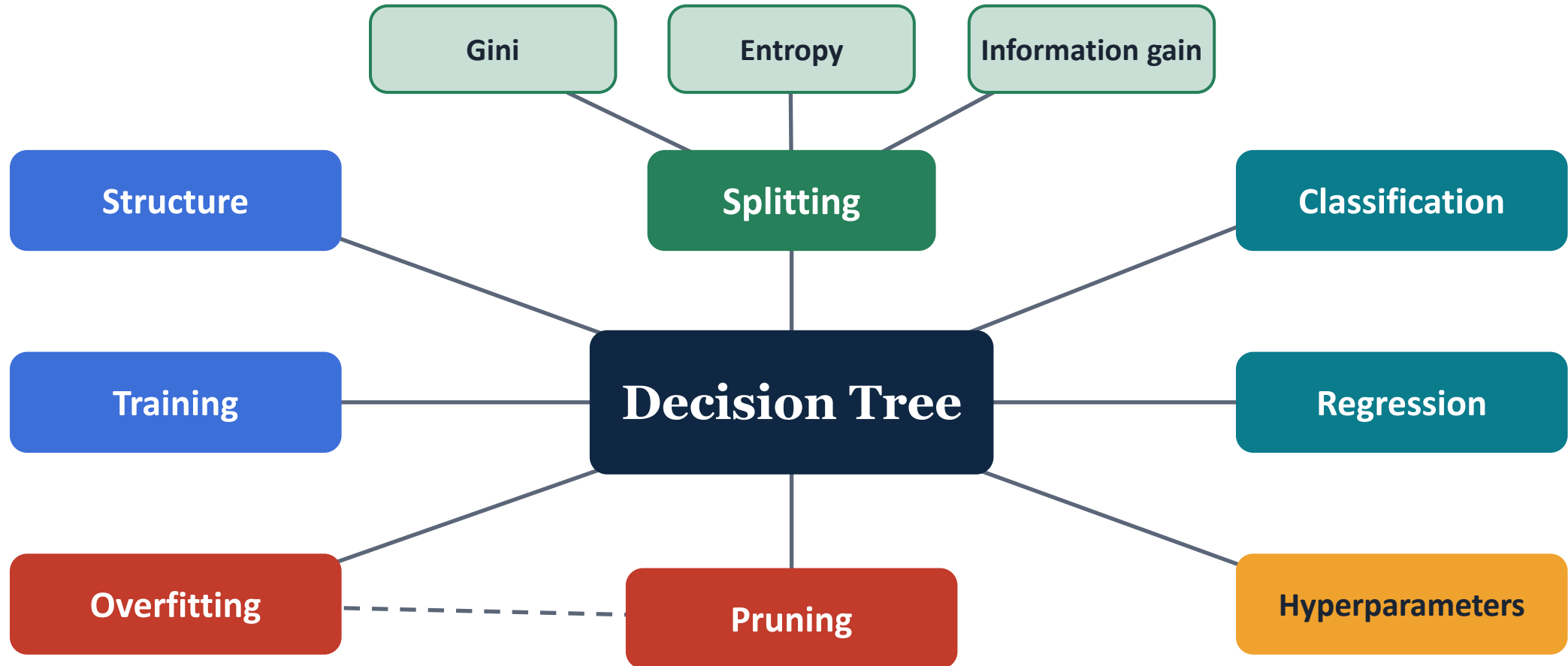
Tune with cross-validation; judge on held-out data

8

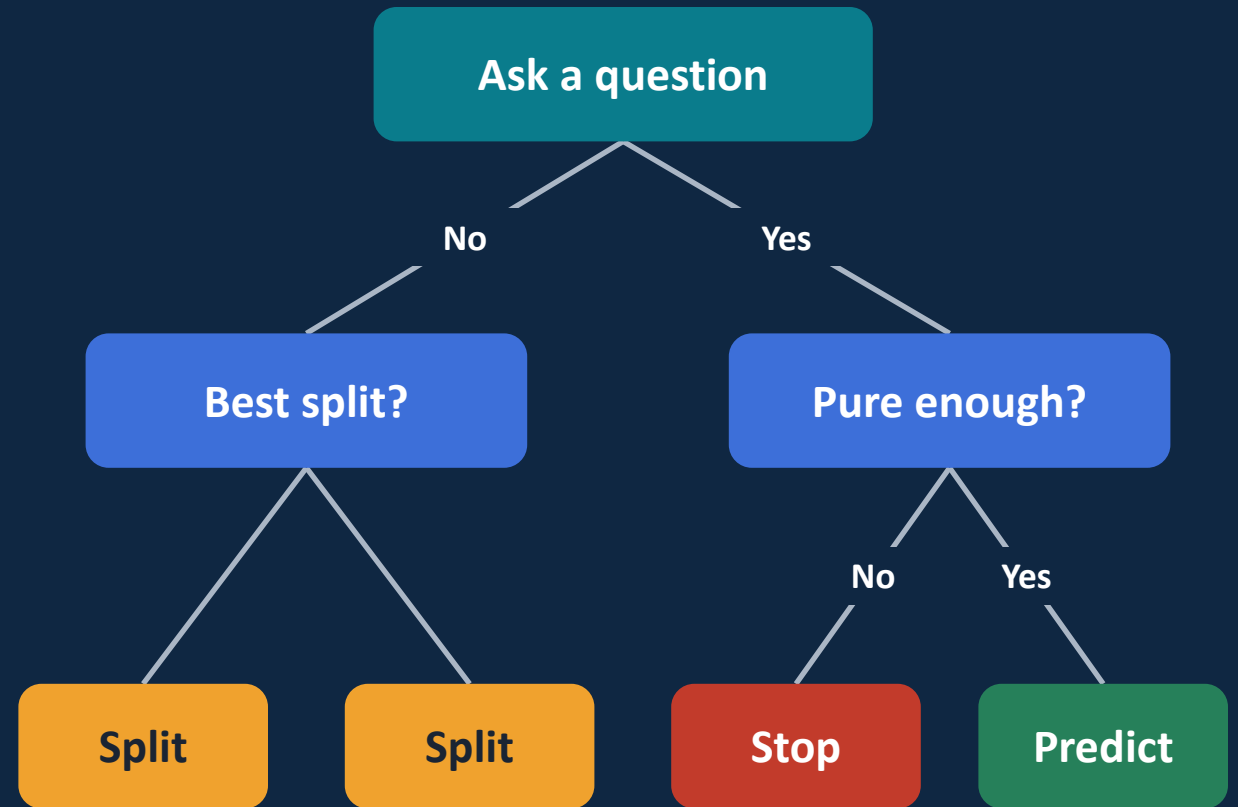
Ensembles such as random forests reduce variance

Concept map

REVIEW



**Think in Questions.
Split the Data.
Make the Decision.**



Questions?